

UNIVERSIDAD NACIONAL DE ROSARIO

FACULTAD DE CIENCIAS EXACTAS, INGENIERÍA Y AGRIMENSURA

Análisis de Lenguajes de Programación

R-322

INFORME TRABAJO PRÁCTICO FINAL

Intérprete para Álgebra Relacional

ALUMNO

Steeman, Juan Ignacio S-5613/8

Febrero, 2026

Índice general

1. Descripción del proyecto	2
1.1. Introducción y Objetivos	2
2. El Lenguaje y su Sintaxis	4
2.1. Diseño de las estructuras principales	4
2.2. Sintaxis Abstracta (BNF)	5
3. Manual de Uso e Instalación	7
3.1. Instalación y Ejecución	7
3.2. Comandos del Intérprete (REPL) y Operaciones del Álgebra Relacional . .	7
3.3. Definición de Operaciones de Usuario	7
3.4. Ejemplo Completo de Sesión	8
4. Organización de los Archivos	9
4.1. Commons.hs	9
4.2. Monads.hs	10
4.3. Parser.y	10
4.4. Eval.hs	11
4.5. Operations.hs	12
4.6. PrettyPrinter.hs	13
4.7. UtilsMain.hs	13
4.8. app/Main.hs	14
4.9. Optimizador.hs	14

1 Descripción del proyecto

1.1. Introducción y Objetivos

El objetivo de este trabajo fue diseñar e implementar un intérprete interactivo del **Álgebra Relacional**. Esta herramienta permite la definición, manipulación y consulta de relaciones.

La arquitectura de procesamiento del programa sigue la siguiente lógica:

Input $\rightarrow$ *Lexer* $\rightarrow$ *Parser* (*AST*) $\rightarrow$ *Optimizador* $\rightarrow$ *Eval* $\rightarrow$ Output (PrettyPrinter)

El intérprete soporta las siguientes **operaciones elementales** del álgebra relacional:

- **Selección** (σ): Filtra tuplas según una condición.
- **Proyección** (π): Selecciona columnas específicas, eliminando los datos redundantes o irrelevantes.
- **Unión** ($\cup$): Devuelve una relación con las tuplas construidas a partir de ambas relaciones.
- **Diferencia** ($-$): Elimina las tuplas de la primera relación que se encuentren en la segunda.
- **Producto Cartesiano** ($\times$): Combina todas las tuplas de dos relaciones.
- **Renombre** (ρ): Renombra atributos para su posterior manejo o para evitar ambigüedades en cruces de datos.

A partir de estas, también implementamos operaciones **derivadas**:

- **Intersección** ($\cap$): Tuplas comunes a ambas relaciones.
- **Natural Join** ($\bowtie$): hace el producto cartesiano y se queda **con las tuplas que tengan valores idénticos** en los atributos comunes, eliminando las que no coinciden.
- **División** ($\div$): Devuelve los valores de los atributos de una relación que están asociados con todos los valores de otra relación.

Además de todo esto, se agregaron dos funcionalidades adicionales que exceden el álgebra relacional clásica:

- **Operaciones definidas por el usuario (macros)**: mediante el comando `:defineOP` es posible parametrizar una expresión y reutilizarla bajo un nombre propio, de forma análoga a una función.

- **Optimizador de consultas:** antes de evaluar una expresión, el intérprete la re-escribe aplicando las optimizaciones clásicas, mejorando el costo de evaluación sin alterar el resultado final.

2 El Lenguaje y su Sintaxis

2.1. Diseño de las estructuras principales

Para la implementación de `Main.hs` se usó como base la del TP de Lambda cálculo; por eso pueden reconocerse en esa estructura varios elementos heredados, como el estado que se lleva a lo largo de toda la ejecución:

```
data State = S{ inter :: Bool ,
                lfile  :: String ,
                ctxt   :: Contex }
```

Otras partes, en cambio, se simplificaron respecto del TP anterior; por ejemplo, el manejo de los archivos.

Para modelar las relaciones utilizamos la siguiente estructura:

```
data Relacion = R {
    nombre      :: String ,
    atributos   :: [(Atributo , Type)] ,
    tuplas      :: Set.Set Tupla
}
```

Esta estructura posee los siguientes campos:

- El **nombre** que sirve de identificador de la relación en el entorno.
- Los **atributos**, que definimos como una lista de tuplas (atributo, tipo) para mantener el orden de los mismos y poder realizar los chequeos de tipos correspondientes.
- Las **tuplas**, que definimos como un `Set` para que no haya repetidos, aprovechando además las funciones que esta estructura nos provee para el manejo de las mismas.

Una tupla tiene la siguiente estructura:

```
type Tupla = Map.Map Atributo Valor
```

Esto asocia el valor (que puede ser nulo, numérico, string o bool) de cada columna con su atributo, garantizando consistencia y un acceso eficiente.

Por último, para distinguir las distintas acciones que el usuario puede ingresar como input definimos:

```
data TopLevel
= TExpr      Expr
| TAssign    NombreRel Expr
| TCmd       Command
deriving Show
```

El rol de TopLevel

La existencia del tipo de datos `TopLevel` responde a una necesidad de separar la semántica pura del álgebra relacional de la lógica de interacción del intérprete (REPL).

Manejo de expresiones (`Expr`). El tipo `Expr` representa los operadores matemáticos del álgebra relacional (selección, proyección, joins, etc.). Evaluar una `Expr` solo produce una nueva `Relacion`, pero no modifica el contexto de las relaciones.

La capa de interacción (`TopLevel`). `TopLevel` es el punto de entrada del intérprete, donde se diferencian tres tipos de acciones fundamentales:

- **Consulta directa (`TExpr`):** evaluar una expresión y mostrar su resultado.
- **Vistas (`TAssign`):** asigna el resultado de una consulta a un nombre en el entorno (creación de vistas).
- **Acciones de entorno (`TCmd`):** comandos del intérprete (ayuda, carga de archivos, limpieza de memoria, etc.).

Esta estructuración así de `TopLevel` nos ayuda en:

- **Un Manejo de estado controlado:** solo los constructores de `TopLevel` que explícitamente lo requieren (`TAssign` o los comandos de `TCmd` como `:createRel`) tienen permiso para modificar el `Context`. Esto evita que la evaluación de una expresión pueda alterar accidentalmente el estado global del sistema.
- **Facilidad de parsing:** permiten que al momento de parsear se pueda identificar rápidamente el modo de operación apenas lee el primer token, evitando ambigüedades entre operadores relacionales y comandos de control.
- **Extensibilidad y modularidad:** si queremos implementar nuevas funcionalidades solo sería necesario extender `TopLevel` o `Command`, al mismo tiempo en que nos divide de forma clara la cada una de las funciones a realizar.

Cabe destacar que, si bien la asignación de vistas (`TAssign`) también modifica el estado del intérprete tal como lo hacen los comandos, se decidió mantenerla separada de `Command` para una mejor interacción con el usuario, pudiendo definir expresiones `X = Y` de manera más natural

2.2. Sintaxis Abstracta (BNF)

A continuación formalizamos la gramática del lenguaje, tal como es procesada por el generador *Happy*.

Dominios básicos:

$$\begin{aligned} \textit{Atributo} &::= \textit{String} \\ \textit{NombreRel} &::= \textit{String} \\ \textit{NombreOp} &::= \textit{String} \\ \textit{Valor} &::= \textit{VInt Int} \mid \textit{VString String} \mid \textit{VBool Bool} \mid \textit{VNull} \\ \textit{Type} &::= \textit{PInt} \mid \textit{PString} \mid \textit{PBool} \end{aligned}$$

Capa de interacción:

$$\begin{aligned} TopLevel ::= & TExpr Expr \\ & | TAssign NombreRel Expr \\ & | TCmd Command \end{aligned}$$

Expresiones algebraicas:

$$\begin{aligned} Expr ::= & ERelacion NombreRel \\ & | ESeleccion Cond Expr \\ & | EProyeccion [Atributo] Expr \\ & | EUnion Expr Expr \\ & | EDiff Expr Expr \\ & | EProd Expr Expr \\ & | ERenombre Atributo Atributo Expr \\ & | EInterseccion Expr Expr \\ & | ENaturalJoin Expr Expr \\ & | EDiv Expr Expr \\ & | ECall NombreOp [Expr] \end{aligned}$$

Condiciones y predicados (σ):

$$\begin{aligned} Cond ::= & PTrue | PFalse \\ & | PEq Atributo Valor | PNeq Atributo Valor \\ & | PLt Atributo Valor | PGt Atributo Valor \\ & | PAttrEq Atributo Atributo \\ & | PAnd Cond Cond | POr Cond Cond \\ & | PNot Cond \end{aligned}$$

Comandos del entorno:

$$\begin{aligned} Command ::= & Quit | Help | Clear | Browse | Reload \\ & | Compile String \\ & | CreateRel NombreRel [(Atributo, Type)] \\ & | InsertRel NombreRel [[Valor]] \\ & | DropRel NombreRel \\ & | DefineOP NombreOp [Atributo] Expr \end{aligned}$$

Notemos que **ECall** es la aplicación de una operación definida por el usuario sobre una lista de argumentos y es la única expresión que no corresponde a un operador clásico del álgebra relacional, sino que es la operación correspondiente con : *defineOp* del capítulo 1.

3 Manual de Uso e Instalación

3.1. Instalación y Ejecución

Para compilar y ejecutar el intérprete utilizamos la herramienta **Stack** para proyectos de Haskell (se debe tener instalada previamente). Se recomienda ejecutar el siguiente comando dentro del directorio raíz del proyecto, para compilar el proyecto:

```
stack build
```

Luego de concluida la compilación, la sesión interactiva (REPL) se inicia con:

```
stack exec TP-AR-exe
```

ó utilizando:

```
stack run
```

Al arrancar, el intérprete se precarga automáticamente el archivo `Ejemplos/relacionesBase.ar`, que define algunas relaciones de ejemplo (`Personas`, `Trabajos`, `Empresas`) listas para usar y probar el interprete desde el primer momento.

3.2. Comandos del Intérprete (REPL) y Operaciones del Álgebra Relacional

Para ver los comandos se puede utilizar el comando `: help` desde la terminal del intérprete.

Las operaciones las veremos en mayor profundidad al abordar el módulo *Operations.hs*.

Nos limitaremos a mostrar solamente algunos ejemplos de uso al final para no ser muy repetitivos.

3.3. Definición de Operaciones de Usuario

El comando `:defineOP` permite parametrizar una expresión y reutilizarla luego como si fuera un operador propio del lenguaje:

```
:defineOP NombreOP (Param1, ..., ParamN) expresion_con_parametros
```

Ejemplo de uso:

```
AR> :defineOP MayoresDe (Rel, Attr, Edad) seleccion [Attr > Edad] (Rel  
    )  
AR> MayoresDe (Personas, edad, 18)
```


Al invocar la operación, el intérprete sustituye cada parámetro por el argumento correspondiente dentro del cuerpo de la expresión, y luego optimiza y evalúa la expresión resultante.

3.4. Ejemplo Completo de Sesión

A continuación, un ejemplo bastante representativo de un archivo de extensión `.ar` (el formato utilizado por `:compile` para compilarlo), en el cual definimos relaciones, insertamos tuplas y creamos vistas (si se quiere ver en mayor profundidad un ejemplo ver el archivo `relacionesBase.ar` de la carpeta Ejemplos):

```
// Creamos la relación Persona:
:createRel Personas nombre:string, edad:int, ciudad:string;

// Insertamos dos tuplas a la relación Personas:
:insertRel Personas (Juan, 25, Rosario), (Ana, 40, SantaFe);

:createRel Trabajos nombre:string, salario:int;
:insertRel Trabajos (Juan, 120000), (Ana, 90000);

// Evaluamos las consultas y las guardamos como vistas:
Adultos = seleccion[edad > 18](Personas);
NombresAdultos = proyeccion[nombre](Adultos);

// Combinamos las relaciones mediante natural join:
Trabajadores = Personas productoNatural Trabajos;
AltosIngresos = seleccion[salario > 100000](Trabajadores);
```

4 Organización de los Archivos

Para la organización de los archivos del proyecto se utilizó la siguiente división:

```
TP-AR
├── app
│   └── Main.hs
├── Ejemplos
│   └── relacionesBase.ar
├── LICENSE
├── package.yaml
├── README.md
├── src
│   ├── Commons.hs
│   ├── Eval.hs
│   ├── Monads.hs
│   ├── Operations.hs
│   ├── Optimizador.hs
│   ├── Parser.hs
│   ├── Parser.y
│   ├── PrettyPrinter.hs
│   └── UtilsMain.hs
└── TP-AR.cabal
```

El núcleo del trabajo práctico se encuentra en la carpeta `/src` y en `app/Main.hs`. A continuación se analiza cada uno de estos archivos.

4.1. Commons.hs

En este módulo se encuentra la base de todo el proyecto. Allí se pueden encontrar principalmente:

- **La representación de los datos:** se definen las estructuras para manejar **relaciones** y **tuplas** (estas últimas representadas como conjuntos, para evitar duplicados y permitir un acceso eficiente).
- **La sintaxis del lenguaje:** se representan las expresiones algebraicas **Expr**, junto con las condiciones de selección **Cond**, que conforman la base de nuestro AST.
- **Los comandos del intérprete:** en **Command** se definen las operaciones permitidas para la interacción con el usuario.
- **El estado del programa:** para llevar el **Context** y el **State**.

- **Los distintos tipos de errores:** para un manejo de excepciones ante entradas inválidas, e informar al usuario del error en particular.

4.2. Monads.hs

Aquí se encuentra la mónada con la que se maneja el estado interno del intérprete:

```
type StateError a = StateT State (Either Error) a
```

`StateError` es un transformador de estado (`StateT`) cuya mónada interna es la mónada de excepciones (`Either`). Esto permite llevar el estado interno del intérprete e informar errores cuando corresponda.

Además, se utiliza la mónada `InputT IO` provista por la librería `Haskeline`, cuyo rol es manejar las interacciones con la consola, y las funciones para obtener y modificar el contexto de las relaciones y operadores nos fueron de gran ayuda.

4.3. Parser.y

El diagrama de flujo del analizador sintáctico es el siguiente:

$$String \rightarrow Lexer \rightarrow [Token] \rightarrow Parser \rightarrow AST \text{ (o Error)}$$

Para implementar el analizador sintáctico de nuestro lenguaje se utilizó, al igual que en el TP2 de λ -cálculo tipado, el generador *Happy*.

Primero se definen los *tokens*, y se realiza un preprocesamiento de la entrada mediante el *lexer* para convertir el string crudo de entrada en una lista de tokens; luego esa lista se transforma en un Árbol de Sintaxis Abstracta (AST).

La estructura del parser se organiza en tres grandes categorías:

- **Comandos del REPL:** vienen con el prefijo “:”, y permiten interactuar con el entorno del intérprete.
- **Vistas:** mediante el símbolo de asignación “=”, permiten guardar el resultado de una consulta para usos futuros.
- **Expresiones:** se parsean todas las expresiones algebraicas definidas en `Commons.hs`, respetando sus precedencias.

Jerarquías y precedencias

Uno de los aspectos más importantes al momento de parsear es poder establecer un orden claro de precedencias entre los operadores. Para las expresiones algebraicas (`Expr`) esto se logra mediante una gramática estratificada en capas (`BinExpr` $\rightarrow$ `JoinExpr` $\rightarrow$ `ProdExpr` $\rightarrow$ `BaseExpr`), de menor a mayor precedencia:

Precedencia	Operadores
Baja	union, diferencia, interseccion
Media	productoNatural
Alta	producto, division
Máxima (términos)	seleccion, proyeccion, renombre, operadores definidos

Tabla 4.1: Precedencia de operadores binarios en el Parser

Para las condiciones (**Cond**), se delega la resolución de la ambigüedad a las directivas de precedencia de Happy:

```
%left or
%left and
%right not
```

Esto resulta en que **and** liga más fuerte que **or**, y **not** (al ser unario y asociar a derecha) liga más fuerte que ambos.

También se permiten comentarios de línea al estilo de C, mediante “//”, útiles principalmente para los scripts de operaciones (**.ar**). El lexer los detecta en la primera pasada sobre el string:

```
| c == '/' && not (null cs) && head cs == '/'
  = lexer (dropWhile (/= '\n') cs)
```

Finalmente, tanto el parser como el resto del intérprete utilizan la mónada **StateError** para el manejo de fallos. Esta estructura permite capturar errores de sintaxis durante la ejecución sin interrumpir el programa de forma abrupta, devolviendo mensajes descriptivos que facilitan la corrección por parte del usuario. Para resolver esto se utiliza la función **parse'** como entrada principal, delegando el manejo final del resultado y la presentación de errores a la función **parse**.

4.4. Eval.hs

Este módulo se encarga de interpretar y ejecutar recursivamente el AST obtenido desde el parser, evaluando las expresiones y devolviendo la relación resultante.

Entre todos los casos analizados, los dos más interesantes son:

- **ERelacion:** es el caso base de la recursión. El evaluador busca el nombre de la relación en el contexto actual: si existe, la devuelve; si no, se lanza un error indicando que la relación no existe.
- **ECall:** se utiliza para manejar las operaciones definidas por el usuario. Para facilitar su comprensión, puede pensarse en cuatro pasos:
 1. Se busca, en el contexto de operaciones definidas por el usuario, la operación solicitada.
 2. Se chequea que la cantidad de argumentos pasados coincida con la cantidad de parámetros de la operación.
 3. Se reemplazan los parámetros de la operación por los argumentos pasados por el usuario, generando un mapa de sustitución y aplicando la función auxiliar **subst**.
 4. Se evalúa la expresión resultante de la sustitución.

Lo que facilitó bastante este trabajo fueron las funciones de la mónada de estado, en particular **getRels** y **getOps**, que dan acceso directo a las relaciones y operaciones definidas en el contexto actual.

4.5. Operations.hs

En este módulo se implementa la semántica de cada una de las operaciones del lenguaje, apoyándonos fuertemente en `Set` y `Map` de las librerías estándar de Haskell.

Operaciones base

Unión ($r \cup s$). Para que la operación sea válida, r y s deben tener el mismo número de atributos, y el dominio del i -ésimo atributo de r debe coincidir con el de s . La relación resultante tiene el mismo esquema que r (o s) y su cuerpo está formado por todas las tuplas de ambas relaciones. Como las tuplas se representan como `Set`, la unión entre relaciones con esquemas compatibles se reduce directamente a la unión de esos conjuntos.

Diferencia ($r - s$). Requiere las mismas condiciones de compatibilidad que la unión. La relación resultante contiene todas las tuplas que están en r pero no en s . Se implementa, análogamente a la unión, mediante la diferencia de conjuntos de tuplas.

Proyección ($\pi_{i_1, \dots, i_k}(r)$). Devuelve una nueva relación restringida a las columnas $i_1, \dots, i_k$. Para implementarla:

1. Se chequea que los atributos a proyectar existan en la relación.
2. Se construye el nuevo esquema, buscando el tipo de cada atributo proyectado.
3. Se restringen las tuplas a los atributos proyectados, utilizando `Map.restrictKeys` sobre cada tupla del conjunto.

Selección ($\sigma_P(r)$). Selecciona todas las tuplas de r que satisfacen el predicado P . Para implementarla se definió una función auxiliar, `filtrarCond`, que evalúa recursivamente la condición P sobre cada tupla y determina si la cumple.

Producto cartesiano ($r \times s$). Combina las tuplas de r con las de s . El principal desafío fue evitar conflictos cuando ambas relaciones comparten atributos con el mismo nombre: antes de combinarlas, se renombran los atributos comunes (tanto en el esquema como en las claves de las tuplas) agregando el sufijo `nombreRelación`". Una vez renombrados, el producto cartesiano se reduce a combinar (mediante `Map.union`) cada tupla de r con cada tupla de s .

Renombre ($\rho_{a \rightarrow b}(r)$). Renombra un atributo a de la relación r por un nuevo nombre b , tanto en el esquema como en cada una de las tuplas, validando previamente que a exista, que b no exista ya en el esquema, y que $a \neq b$.

Operaciones derivadas

Intersección ($r \cap s$). Exige la misma compatibilidad de esquemas que la unión y la diferencia. Se implementa en términos de las operaciones base: $r \cap s = r - (r - s)$.

Natural Join ($r \bowtie s$). Conceptualmente equivale a realizar el producto cartesiano entre r y s , seleccionar únicamente las combinaciones cuyos atributos comunes coinciden, y proyectar eliminando las columnas duplicadas:

$$r \bowtie s = \pi_{R \cup S}(\sigma_{r.A_1=s.A_1 \wedge \dots \wedge r.A_n=s.A_n}(r \times s)), \quad \{A_1, \dots, A_n\} = R \cap S$$

La implementación, sin embargo, no pasa literalmente por el producto cartesiano y la selección por separado, sino que combina ambos pasos en una sola pasada para mejorar su eficiencia:

1. Se determinan los atributos comunes entre los esquemas de r y s .
2. Se define el esquema resultante como la unión de ambos esquemas, sin atributos repetidos.
3. Para cada par de tuplas (una de r , una de s), se verifica que coincidan los valores de todos los atributos comunes.
4. Las tuplas compatibles se combinan mediante `Map.union`, conformando el cuerpo de la relación resultante.

División ($r \div s$). Sean $r(X_1, \dots, X_m, Y_1, \dots, Y_n)$ y $s(Y_1, \dots, Y_n)$, con $X = X_1, \dots, X_m$ e $Y = Y_1, \dots, Y_n$. La relación $r \div s$ tiene esquema X , y su cuerpo está formado por todas las tuplas x tales que (x, y) pertenece a r para *todo* y de s . Para implementarla se sigue la formulación clásica $r \div s = \pi_X(r) - \pi_X((\pi_X(r) \times s) - r)$:

1. Se valida que los atributos de s (el conjunto Y) sean un subconjunto de los atributos de r .
2. Se calcula X como los atributos de r que no pertenecen a s , y se proyecta r sobre X (obteniendo los posibles candidatos).
3. Se calcula el producto cartesiano entre esa proyección y s , generando todas las combinaciones (x, y) posibles.
4. Se resta r a ese producto: lo que queda son las combinaciones (x, y) que *no* están en r , es decir, los candidatos x que fallan para al menos un y de s .
5. Se proyecta esa diferencia sobre X , obteniendo los candidatos inválidos.
6. Finalmente, se restan los candidatos inválidos a $\pi_X(r)$, quedando solo los x que sí están asociados a *todos* los valores de s .

4.6. PrettyPrinter.hs

Aquí se definen las funciones para mostrar las relaciones, expresiones y condiciones de forma legible. La función principal es `prettyRelacion`, que imprime una relación con formato de tabla (encabezado con los nombres de los atributos, separadores y una fila por tupla). También se incluyen `prettyExpr` y `prettyCond`, que reconstruyen la notación matemática habitual del álgebra relacional (σ , π , ρ , $\bowtie$, etc.) a partir del AST; y `prettyContext`, que lista las relaciones y operaciones definidas en el entorno (utilizada por el comando `:browse`).

4.7. UtilsMain.hs

Aquí se implementan las funciones que ejecutan cada comando del REPL (definir operaciones, crear y eliminar relaciones, insertar tuplas, realizar vistas, ejecutar expresiones, entre otras), así como la manipulación de archivos (`:compile`, `:reload`). Es también en este módulo donde se encadenan el optimizador y el evaluador: tanto al ejecutar una expresión suelta como al definir una vista, primero se optimiza la expresión con `optimizador` y luego se evalúa el resultado con `evalExpr`.

4.8. app/Main.hs

Para este módulo se tomó como base el desarrollado en el TP2 de Lambda cálculo simplemente tipado. Se inicializa la sesión interactiva utilizando la librería `Haskeline`, y se capturan las excepciones de entrada/salida (como el fin de archivo o una interrupción del usuario) para finalizar la ejecución de forma controlada en lugar de abortar abruptamente.

Al iniciar el sistema se precarga el archivo prelude (`Ejemplos/relacionesBase.ar`), además de los archivos que se hayan pasado como argumentos por línea de comandos; luego se imprime la bienvenida y se entra en el ciclo iterativo de lectura, evaluación e impresión administrado por la función `readevalprint`.

4.9. Optimizador.hs

Este fue el último módulo implementado, y su objetivo es reducir el costo de evaluación de una consulta reescribiendo el AST antes de evaluarlo, sin alterar el resultado final. Las reescrituras realizadas fueron:

- **Expansión de macros (ECall):** antes de optimizar una llamada a una operación definida por el usuario, se sustituyen sus parámetros por los argumentos recibidos y se optimiza recursivamente la expresión ya expandida, de forma que las optimizaciones siguientes también se aplican dentro del cuerpo de la macro.
- **Empuje de selecciones sobre productos y joins:** una selección sobre un producto cartesiano o un join natural se empuja hacia la subexpresión que efectivamente contiene los atributos involucrados en la condición, reduciendo la cantidad de tuplas combinadas innecesariamente.
- **Fusión de selecciones consecutivas:** `seleccion[c0](seleccion[c1](e))` se reescribe como una única selección con la condición `c0 and c1`.
- **Eliminación de proyecciones redundantes:** una proyección sobre otra proyección se simplifica, conservando solo la externa.
- **Forma normal conjuntiva (FNC):** las condiciones de las selecciones se normalizan distribuyendo el `or` sobre el `and`.
- El resto de las operaciones (uniones, diferencias, intersecciones, divisiones, renombres) se optimizan recursivamente, propagando la optimización a sus subexpresiones.

Una cosa importante para aclarar es que el manejo de errores semánticos (operación inexistente, cantidad errónea de argumentos, entre otros) es deliberadamente diferido al evaluador para centralizar la validación en un único punto y no tener que manejar los errores en distintos lados innecesariamente.

Bibliografía

- [1] Wikipedia. *Álgebra Relacional*. Disponible en: https://en.wikipedia.org/wiki/Relational_algebra
- [2] Slides de la cátedra de Teoría de Base de Datos. *Álgebra Relacional*. Disponible en: <https://campusv.fceia.unr.edu.ar/>
- [3] Hackage. *Documentación de Happy*. Disponible en: <https://haskell-happy.readthedocs.io/en/latest/using.html>